

PentAGI 工作原理与架构分析

分析日期：2026-05-09 | 仓库：github.com/vxcontrol/pentagi | Stars: 16.6k

1. 项目定位

PentAGI（**P**enetration testing **A**rtificial **G**eneral **I**ntelligence）是一个**开源、自托管的自主渗透测试系统**。后端使用 Go 语言，前端 React + TypeScript，采用多 AI Agent 协作完成复杂的安全测试任务。系统集成 20+ 专业安全工具（nmap、Metasploit、sqlmap 等），所有操作在沙箱化 Docker 容器中执行。

核心定位：面向信息安全专业人士、研究者和爱好者，提供完全自主可控的 AI 驱动渗透测试方案。自托管部署，用户对部署和数据拥有完全控制权。

2. 整体架构

系统采用**微服务架构**，支持水平扩展。核心组件通过 Docker Compose 编排，分为三个独立网络层级：

层级	组件	职责
核心服务层	React UI + Go API + PostgreSQL/pgvector + 任务队列	用户交互、API 网关、数据持久化、异步处理
知识图谱层	Neo4j + Graphiti API	语义关系追踪，捕获 Agent 响应和工具执行记录
监控层	Grafana + VictoriaMetrics + Jaeger + Loki	OpenTelemetry 全链路可观测性
分析层	Langfuse + ClickHouse + Redis + MinIO	LLM 可观测性、分析数据仓库、缓存、对象存储

3. 多 Agent 协作系统（核心架构）

3.1 workflow 层级模型

```
Flow (顶层渗透测试活动)
├── Task (任务)
│   ├── SubTask (子任务, 绑定特定 Agent 类型)
│   │   ├── Action (具体动作: 命令/搜索/分析/执行)
│   │   │   ├── Artifact (产物: 文件/报告/日志/截图)
│   │   │   └── Memory (记忆: 观察/结论 + 向量嵌入)
```

3.2 四阶段 Agent 协作流程

这是 PentAGI 最核心的设计 — **Orchestrator（编排器）** 协调三类专业 Agent 完成渗透测试全流程：

Phase 1 — 研究阶段（Researcher Agent）

- Orchestrator 将目标信息发送给 Researcher
- Researcher 搜索类似案例、查询已知漏洞（CVE）
- 将发现存入向量存储（pgvector），返回研究结果

Phase 2 — 规划阶段（Developer Agent）

- Orchestrator 将研究结果传给 Developer
- Developer 查询 Exploit 数据库（Sploit.us 等）和工具信息
- 生成**结构化攻击计划**返回给 Orchestrator

Phase 3 — 执行阶段（Executor / Pentester Agent）

- Orchestrator 将攻击计划发送给 Executor
- Executor 加载工具使用指南（Tool Guides），执行具体攻击步骤
- 结果和状态持续回传，存入知识图谱（Graphiti + Neo4j）

全程并行支撑系统

- **Vector Store** (pgvector) — 提供历史案例的语义搜索和相似任务匹配
- **Knowledge Base** (Neo4j 图数据库) — 存储领域知识、工具能力和实体关系

3.3 Agent 类型清单（11 种）

Agent	角色定位	工具调用上限
Assistant	主编排器，管理 Flow 生命周期	100（通用级）
Pentester	渗透执行，运行安全工具	100（通用级）
Coder	代码生成/漏洞分析/Exploit 开发	100（通用级）
Installer	环境和依赖安装配置	100（通用级）
Searcher	多源信息搜索（Google、DuckDuckGo、Sploitius 等）	20（受限级）
Adviser (Mentor)	执行监督，检测死循环和异常行为	20（受限级）
Planner	任务规划，生成结构化执行步骤	20（受限级）
Memorist	记忆管理，写入/检索向量存储	20（受限级）
Reflector	反思纠错，处理 LLM 调用失败	20（受限级）
Generator	报告和-content生成	20（受限级）
Enricher	上下文增强，分析执行环境	20（受限级）

4. 四层监督机制（核心创新）

第 1 层：执行监控 — Mentor/Adviser（Beta）

- 自动检测**连续相同工具调用**（阈值: 5 次）和**总调用次数超限**（阈值: 10 次）
- Adviser Agent 被自动激活，评估执行进度，检测 Agent 是否陷入死循环
- 工具响应中注入 `<mentor_analysis>` 段落，提供改进建议
- 环境变量开关： `EXECUTION_MONITOR_ENABLED` （默认关闭）
- **代价**：2-3 倍执行时间和 Token 消耗 | **收益**：结果质量提升 2 倍

第 2 层：智能任务规划 — Planner (Beta)

- 在专家 Agent 启动前，Planner 生成 **3-7 个具体可执行步骤**
- Enricher Agent 先分析完整执行上下文
- 原始请求包装在 `<task_assignment>` XML 结构中，附带执行计划
- **防止 Agent 偏离目标 (scope creep)**
- 环境变量开关： `AGENT_PLANNING_STEP_ENABLED` (默认关闭)

第 3 层：硬性工具调用限制 (始终开启)

- 通用 Agent (Assistant/Pentester/Coder/Installer) : 默认 100 次上限
- 受限 Agent (其余) : 默认 20 次上限
- 接近上限 (剩余 ≤ 3 次) 时触发 Reflector 引导优雅终止

第 4 层：Reflector 反思机制 (始终开启)

- LLM **连续 3 次** 未能生成有效工具调用时触发
- 分析失败原因，引导 Agent 改用 `done` (标记完成) 或 `ask` (向用户提问) 等屏障工具
- 具备**递归保护** (`isReflectorRetry` / `markReflectorRetry` 标志位)，防止无限循环
- 协调在工具调用限制时的优雅终止

官方测试结论： 对于参数量 32B 以下的开源小模型（如 Qwen3.5-27B-FP8），同时启用执行监控和任务规划后，**结果质量相比无监督基线提升 2 倍**。官方明确推荐小模型部署时默认开启这两项功能。

5. Agent 执行循环 (performAgentChain)

核心执行引擎位于后端 `backend/pkg/providers/performer.go`，是一个多轮迭代的 Agent 循环：

```
loop (最多 100/20 次迭代) :  
|
```

```
├─ 1. callWithRetries(LLM)
│   ├── 调用 LLM, 流式获取响应
│   ├── 失败 → 等待 5s → 重试 (最多 3 次)
│   ├── 全部失败 → performReflector(反思) → 重试
│   └── 成功 → 返回 tool_calls 列表
│
├─ 2. execToolCall (每个 tool_call)
│   ├── 重复检测器 → 连续相同调用超阈值 → 警告/中止
│   ├── 执行工具 → 失败 → fixToolCallArgs(修复参数) → 重试(最多3次)
│   ├── Mentor 监督 → 达到监控阈值 → 注入 mentor_analysis
│   └── 工具结果注入消息链
│
├─ 3. 流式推送 → reasoning/text chunks → 前端实时展示
│
├─ 4. Summarizer → 压缩消息链 → 防止 Token 溢出
│
├─ 5. 持久化 → 消息链 JSON → PostgreSQL + Token/Cost 记录
│
├─ 6. Graphiti → 可选 → 工具执行和 Agent 响应存入 Neo4j
│
├─ 7. 检查终止条件
│   ├── done/ask 工具被调用 → 正常退出
│   ├── 迭代次数接近上限 → Reflector 引导退出
│   └── 重复检测中止 → 异常退出
│
└─ 回到 loop
```

6. 记忆系统架构

记忆类型	存储介质	内容	作用
长期记忆	pgvector (向量数据库)	嵌入向量、领域知识、工具使用模式	跨会话知识复用
工作记忆	当前会话上下文	任务状态、活动目标、系统资源	当前任务状态追踪
情景记忆	历史记录 + Neo4j	过去命令序列、执行结果、成功模式	经验积累和模式识别

三种记忆形成**反馈闭环**:

```
上下文查询 → 向量库检索相似任务
目标规划 → 知识库提供领域知识
动作执行 → 产生结果/模式
模式提取 → 反哺向量库和知识图谱
```

7. 消息链压缩系统（Chain Summarization）

独创的 **ChainAST**（消息链抽象语法树）结构，在压缩时保留消息类型信息（包括工具调用的结构化内容）：

压缩流程：

1. 检查是否需要压缩（Token 使用率超阈值）

2. 将消息链转换为 ChainAST（结构化表示）

3. 分段摘要（Section Summarization）

4. 处理超大消息对（Oversized Pairs）

5. 管理尾部保留段（默认保留最后 N 条不压缩）

6. QA 摘要（保留最近 7-10 个 QA 交互对）

7. 重建压缩后的消息链

8. 校验：如果压缩后反而更大 → 返回原始链

参数	全局默认	Assistant 专用
保留尾部	PRESERVE_LAST=true	同全局
尾部保留大小	50KB	75KB（更多记忆空间）
最大 QA 段数	10	7
QA 段最大字节	64KB	75KB

8. LLM 提供商抽象层

`backend/pkg/providers/` 下 14 个子目录，每个实现统一的 Provider 接口：

提供商	模型数	特性
OpenAI	31	GPT-5.2/5.1/4.1/4o + o1/o3/o4-mini reasoning
Anthropic	10	Opus 4.6 / Sonnet 4.6 / Haiku 4.5 + legacy
Google Gemini	13	Gemini 3.1/3/2.5/2.0 + Gemma 开源
AWS Bedrock	21	Nova/Claude/Cohere/DeepSeek/Qwen/Mistral
DeepSeek	2	V3.2 Chat + Reasoner
GLM (智谱)	12	MoE 架构, GLM-5 到 GLM-4
Ollama	—	本地推理 + Ollama Cloud
聚合器	—	OpenRouter / DeepInfra / LiteLLM Proxy

统一 Reasoning 处理：所有 Provider 的推理内容通过 `TextPartWithReasoning` 统一封装，实现跨 Provider 的 reasoning 去重和兼容。

9. 后端代码组织

包路径	文件数	职责
<code>pkg/controller/</code>	22	实体 CRUD + 9 种日志通道 (alog/aslog/msglog/slog/termlog/vslog...)
<code>pkg/providers/</code>	14 子目录 + 10 文件	LLM 抽象层 + Agent 执行引擎
<code>pkg/tools/</code>	35	工具系统：搜索、浏览器、终端、代码、记忆，注册表模式
<code>pkg/graphiti/</code>	—	Neo4j 知识图谱集成
<code>pkg/database/</code>	—	PostgreSQL 数据访问层
<code>pkg/server/</code>	—	REST + GraphQL API 服务
<code>pkg/queue/</code>	—	异步任务队列
<code>pkg/docker/</code>	—	Docker 沙箱编排
<code>pkg/terminal/</code>	—	终端模拟

10. 安全架构

- **沙箱化执行**：所有操作在独立 Docker 容器中运行
- **网络隔离**：核心服务、监控、分析各自独立网络
- **双节点部署（生产推荐）**：Worker 节点独立硬件 + Docker-in-Docker + TLS 认证
- **API 认证**：Bearer Token，支持配置过期时间（1 分钟 ~ 3 年）
- **SSL 支持**：默认自签名证书，支持自定义证书

总结

PentAGI 的本质设计理念是：

层级化多 Agent 协作 + 多层监督反思机制 + 结构化记忆系统 + 沙箱安全隔离

它把渗透测试这个复杂、开放式的安全问题，分解为可管理的子任务链。其真正的工程创新不在于单个 Agent 的能力，而在于：

1. **Agent 之间的编排体系**（Researcher → Developer → Executor）
2. **四层监督纠错机制**（Mentor/Planner/Limits/Reflector）
3. **ChainAST 消息压缩**对长对话上下文管理的工程化处理
4. **三类型记忆系统的反馈闭环**（长期/工作/情景记忆联动）